

Лекция 13

Библиотека *cuBLAS*.

- Особенности использования библиотеки cuBLAS.
- Функции `cublas<T>gemm()`.
- Реализация произведения матриц на основе cuBLAS.
- Тензорные процессоры.
- Вызовы `cublas<T>gemm()` с использованием тензорных процессоров.

Библиотека cuBLAS

(*Basic Linear Algebra Subroutines*)

- Хранение по столбцам (column-major storage), совместимость с Фортраном, для копирования и инициализации матриц следует использовать специальный API.
- Линейная индексация массивов;
- Имена функций образуются по схеме: cublas<T><function>. Например, **cublasSgemm**, тип данных *float*, *generic/general matrix-matrix* умножение плюс сложение: $C = \alpha AB + \beta C$

Документация: [**https://docs.nvidia.com/cuda/cublas/#**](https://docs.nvidia.com/cuda/cublas/#)

```
int main(){
```

```
// Инициализация библиотеки CUBLAS
```

```
cublasHandle_t cublas_handle;  
cublasCreate(&cublas_handle);
```

leading
dimension

```
.....  
//Копирование матрицы с числом строк num_rows и числом  
столбцов //num_cols с хоста на устройство
```

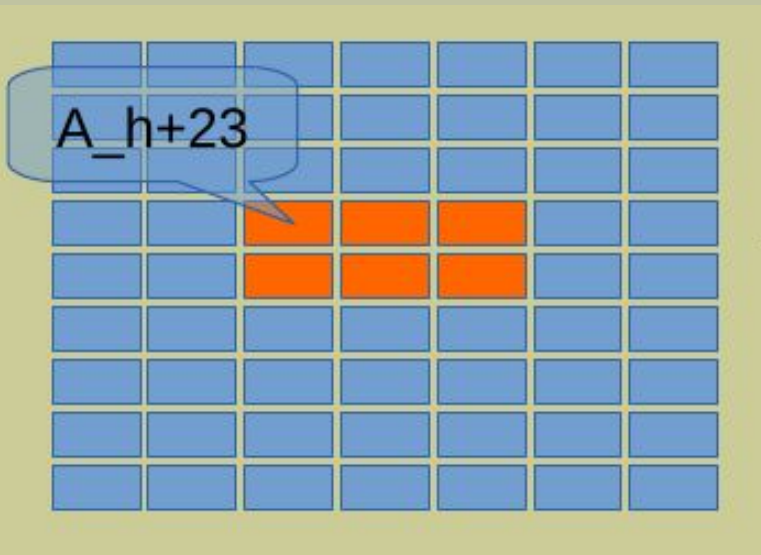
```
cublasSetMatrix(num_rows, num_cols, elem_size, A_h, Idah, A_dev, Idad);
```

```
<вызов функции cuBLAS API>
```

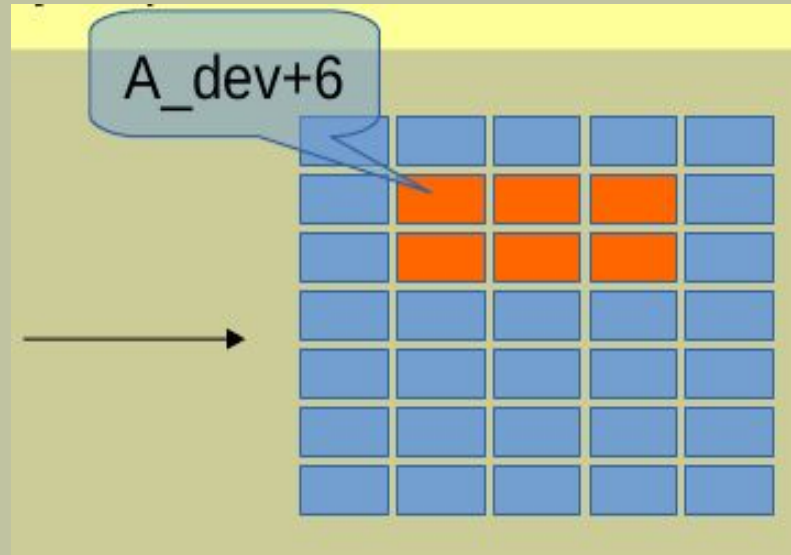
```
cublasGetMatrix(num_rows, num_cols, elem_size, A_dev, num_rows, A_h,  
num_rows);
```

```
.....  
cublasDestroy(cublas_handle);
```

```
}
```



leading dimension равно 9



leading dimension равно 7

```
void hMultMatsBlas(REAL* A, REAL* B, REAL* C){  
    cublasHandle_t cublas_handle;  
    cublasCreate(&cublas_handle);  
  
    const float alpha=1.0;  
    const float beta=0.0;  
    // cublasSetMathMode(cublas_handle, CUBLAS_TENSOR_OP_MATH);  
    cublasSgemm(cublas_handle, CUBLAS_OP_T, CUBLAS_OP_N,  
                M, N, K,  
                &alpha,  
                A, M,  
                B, K,  
                &beta,  
                C, M);  
    cublasDestroy(cublas_handle);  
}
```

```
void hMatOutBlas(REAL* D, int I, int J){  
    REAL* Dh=(REAL*)calloc(I*J, sizeof(REAL));  
    cublasGetMatrix(I, J, sizeof(REAL), D, I, Dh, I);  
    .....  
}
```

“NVIDIA GeForce RTX 2060”

Ускорение по отношению к алгоритму на основе *CUDA API* на *GPU 7.68*.

```
~/Lecture8/Lab8blas> nvprof ./lab8blas
```

```
16.51% 516.96us   1  516.96us 516.96us 516.96us
                                volta_sgemm_128x64_tn
3.81% 119.39us   2  59.695us 59.552us 59.839us
                                glnit(float*, int)
```

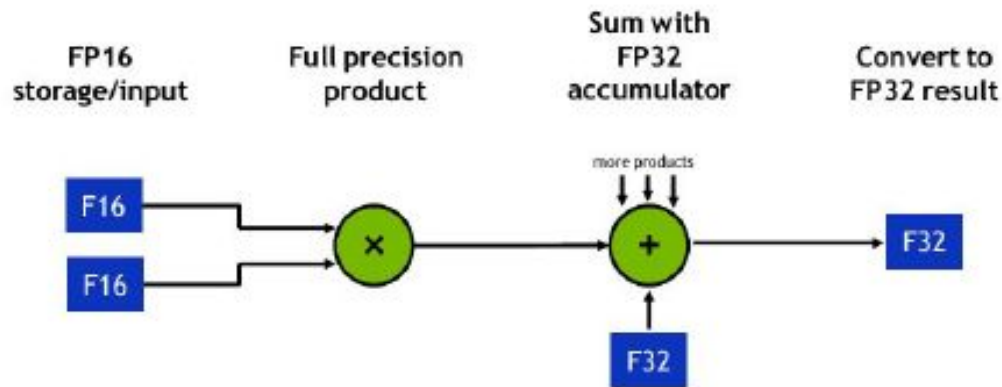
“GeForce GTX 1050”

```
~/WORKSHOP/PGP-2023> nvprof ./lab8blas
```

```
.....
41.49% 1.3640ms   1  1.3640ms 1.3640ms 1.3640ms
                                sgemm_128x128x8_NT_vec
9.28% 305.00us   2  152.50us 152.36us 152.65us
                                glnit(float*, int)
```

Нейронные процессоры:

- специализированные микросхемы, “заточенные” на ускорение алгоритмов машинного обучения;
- аппаратная реализация GEMM ($C = \alpha AB + \beta C$);
- вычисления со смешанной точностью (FP16, FP32, FP64);
- перемножение матриц размерности 4x4, 8x8, 16x16 за один такт;



*GETTING STARTED WITH
TENSOR CORES IN HPC*
Vishal Mehta, NVIDIA
Super Computing, 2019

Google TPU,
Huawei Ascend 310 / Ascend 910,

.....

NVIDIA Tensor Cores.

V100: 64 GEMM за такт, архитектура Volta (Nvidia Tesla V100), sm_70;
TU100-104: архитектура Turing (Geforce RTX 20*), sm_75;
A100: 256 GEMM за такт, архитектура Ampere (Geforce RTX 30*), sm_80.

WMMA, <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#wmma>;
cuBLAS, <https://docs.nvidia.com/cuda/cublas/>;
CUTLASS, <https://nvidia.github.io/cutlass/>;
cuTENSOR, <https://docs.nvidia.com/cuda/cutensor/>;
cuDNN, <https://developer.nvidia.com/cudnn>;
TensorFlow, <https://www.tensorflow.org/>,
<https://docs.nvidia.com/deeplearning/frameworks/tensorflow-user-guide/>;
PyTorch, <https://pytorch.org/>.

1. CUTENSOR A CUDA Library for High-Performance Tensor Primitives, Paul Springer, November 20th 2019.
2. GETTING STARTED WITH TENSOR CORES IN HPC, Vishal Mehta, NVIDIA Super Computing, 2019
3. CUTLASS: CUDA TEMPLATE LIBRARY FOR DENSE LINEAR ALGEBRA AT ALL LEVELS AND SCALES, Jeng Bai-Cheng (Ryan), 21 Nov.
4. VOLTA TENSOR CORE TRAINING ORNL, August 2019.
5. Stefano Markidis et al. NVIDIA Tensor Core Programmability, Performance & Precision, arXiv:1803.04014 [cs.DC], (2018). <https://doi.org/10.48550/arXiv.1803.04014>.

Тензорные ядра не активны.

```
~/Lecture8/Lab8blas> ncu --replay-mode application --metrics  
sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active lab8blas
```

```
volta_sgemm_128x64_tn, 2023-Mar-20 13:56:45, Context 1, Stream 7
```

Section: Command line profiler metrics

```
-----  
sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active    %      0  
-----
```

```
~/Lecture8/Lab8blas> nvprof ./lab8blas
```

```
.....  
11.63% 308.32us 1 308.32us 308.32us 308.32us  
                               volta_s884gemm_128x128_ldg8_f2f_tn  
4.44% 117.79us 2 58.895us 58.784us 59.007us  
                               glnit(float*, int)  
.....
```

Тензорные ядра активны.

```
~/Lecture8/Lab8blas> ncu --replay-mode application --metrics
```

```
sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active lab8blas
```

```
volta_s884gemm_128x128_ldg8_f2f_tn, 2023-Mar-20 12:29:27, Context 1, Stream 7  
Section: Command line profiler metrics
```

```
-----  
sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active  %      37.90  
-----
```

Ускорение по отношению к алгоритму на основе *CUDA API* на *GPU* 12.9.

Потеря точности.

5.23777	5.23777		5.23777
1347.41	1347.41		1347.41
2689.56	2689.56		2689.56
.....			
9400.73	9400.73		9400.73

```
void hMultMatsBlas(half* A, half* B, half* C){
```

```
.....  
const half alpha=1.0;
```

```
const half beta=0.0;
```

```
  
cublasSetMathMode(cublas_handle, CUBLAS_TENSOR_OP_MATH);
```

```
cublasHgemm(cublas_handle, CUBLAS_OP_T, CUBLAS_OP_N,
```

```
            M, N, K,
```

```
            &alpha,
```

```
            A, M,
```

```
            B, K,
```

```
            &beta,
```

```
            C, M);
```

```
cublasDestroy(cublas_handle);
```

```
}
```

```
__global__ void gInit(REAL* D, int s){
```

```
.....  
    D[j+i*J]=__float2half(s*(float)((j+i*J)*1.0E-5)+(1-s)*1.0f);  
}
```

```
int main(){
```

```
    half *A, *B, *C;
```

```
    cudaMalloc((void**)&A, M*K*sizeof(half));
```

```
.....  
    hMultMatsBlas(A,B,C);
```

```
.....  
}
```

```
.....  
fprintf(stdout, "%g\t", __half2float(Dh[i+j*I]));
```

```
.....
```

```
/Lecture8/Lab8blas> nvprof ./lab8blasH
13.92% 118.59us 2 59.295us 59.231us 59.359us
                        glnit(__half*, int)
11.68% 99.519us 1 99.519us 99.519us 99.519us
                        turing_h1688gemm_128x128_ldg8_stages_32x1_tn
```

```
~/Lecture8/Lab8blas> ncu --replay-mode application --metrics
sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active
lab8blas
turing_h1688gemm_128x128_ldg8_stages_32x1_tn, 2023-Mar-20 14:42:07, Context
1, Stream 7
```

Section: Command line profiler metrics

```
-----
sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_active    %    62.27
-----
```

Тензорные ядра активны.

Потеря точности.

5.23828	5.23828.....	5.23828
1344	1344.....	.1344
2688	2688.....	.2688
.....		
9392	9392.....	.9392

Ускорение по отношению к алгоритму на основе CUDA API на GPU
39.9.

Ускорение по отношению к алгоритму на CPU **39992.36.**

В сорок тысяч раз!

ЗАДАНИЕ.

Реализовать вычисление произведения матриц на GPU, используя CUDA API (“сырой код”) и, отдельно, используя библиотеку *cuBLAS*. Сравнить время выполнения программ при различной размерности матриц.